

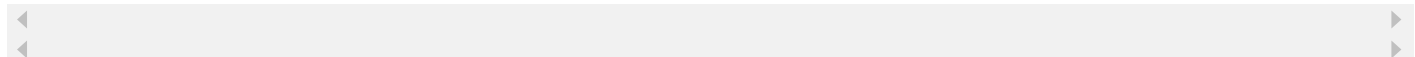
The XOR (Exclusive OR) problem is a legendary milestone in AI history. In 1969, Minsky and Papert famously proved that a single-layer Perceptron could not solve XOR because the data is not **linearly separable**. This nearly killed neural network research for a decade—until **Backpropagation** proved that adding a hidden layer could solve the "unsolvable."

Lecture Notes: Solving the XOR Problem

1. The Problem: Linear Inseparability

The XOR function returns **1** only if the inputs are different, and **0** if they are the same.

Input x_1	Input x_2
0	0
0	1
1	0
1	1



If you plot these points on a 2D graph, you cannot draw a single straight line to separate the "0"s from the "1"s. To solve this, we need to warp the coordinate space using a **Hidden Layer**.

2. The Architecture

To solve XOR, we use a Multilayer Perceptron (MLP):

- **Input Layer:** 2 neurons (x_1, x_2)
 - **Hidden Layer:** 2 neurons (h_1, h_2) with Sigmoid activation.
 - **Output Layer:** 1 neuron ($\hat{y}$) with Sigmoid activation.
-

3. The Training Example (One Step of Backprop)

Let's trace one training iteration using the input $(1, 0)$ where the target is $y = 1$.

Phase 1: Forward Pass

Assume some initial weights (simplified for this example):

- Weights to Hidden Layer: $W_h = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$
 - Weights to Output Layer: $W_o = [0.5 \quad -0.5]$
- Calculate Hidden Neurons ($z \rightarrow a$):**
 $z_{h1} = (1 \cdot 0.5) + (0 \cdot 0.5) = 0.5 \Rightarrow a_{h1} = \sigma(0.5) \approx 0.62$
 $z_{h2} = (1 \cdot 0.5) + (0 \cdot 0.5) = 0.5 \Rightarrow a_{h2} = \sigma(0.5) \approx 0.62$
 - Calculate Output Neuron:**
 $z_o = (0.62 \cdot 0.5) + (0.62 \cdot -0.5) = 0$
 $\hat{y} = \sigma(0) = 0.5$

The Error: Our target is 1, but we predicted 0.5.

Phase 2: The Backward Pass (The Magic)

We now calculate how to change the weights to make $\hat{y}$ closer to 1.

Step 1: Output Error Gradient

We find the gradient of the Loss with respect to the output weight (w_o).

$$\delta_o = (\hat{y} - y) \cdot \sigma'(z_o)$$

Since $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$\delta_o = (0.5 - 1) \cdot (0.5 \cdot 0.5) = -0.125$$

Step 2: Hidden Layer Error Gradient

Now we pass that error back to the hidden neurons. The "blame" for h_1 is the output error multiplied by the weight connecting h_1 to the output.

$$\delta_{h1} = (\delta_o \cdot w_{o1}) \cdot \sigma'(z_{h1})$$

$$\delta_{h1} = (-0.125 \cdot 0.5) \cdot (0.62 \cdot 0.38) \approx -0.0147$$

Step 3: Weight Updates

Using a learning rate $\alpha = 0.1$:

$$w_{new} = w_{old} - (\alpha \cdot \text{Gradient})$$

- For Output Weight:** $0.5 - (0.1 \cdot -0.125 \cdot 0.62) \approx 0.507$
 - For Hidden Weight:** $0.5 - (0.1 \cdot -0.0147 \cdot 1) \approx 0.501$
-

4. How XOR is Actually Solved

During hundreds of these iterations, the Backpropagation algorithm "stretches" the data.

The hidden layer learns to recognize features. For example:

- h_1 might learn to fire if **either** input is 1 (an **OR** gate).

- h_2 might learn to fire if **both** inputs are 1 (an **AND** gate).

The output neuron then simply subtracts the "AND" signal from the "OR" signal. By the end of training, the hidden layer has transformed the inseparable XOR points into a new 2D space where a single straight line *can* separate them.

Summary of Concepts

- **Hidden Layers:** Allow the network to create new internal representations of data.
- **Chain Rule:** Transfers the error from the output layer back to the very first weight in the network.
- **Non-Linearity:** Without Sigmoid/ReLU, the hidden layer couldn't "warp" the space to solve the inseparability.